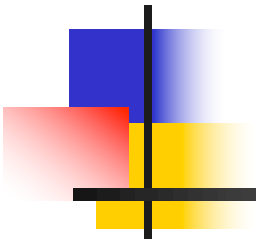


7.3 Caso de uso “ver detalles de un producto”



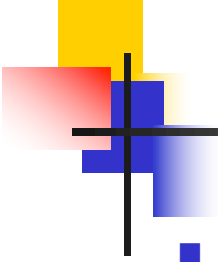
- Componentes
- Hook useEffect

Componentes

- El enlace a cada producto que aparece en el resultado de una búsqueda se imprime desde **Products** con

```
<ProductLink id={product.id} name={product.name}/>
```

ProductLink		Products
Department	Name	
Movies	2001: A Space Odyssey [Blu-ray]	
Movies	Alien [DVD]	
Music	Animals [CD]	

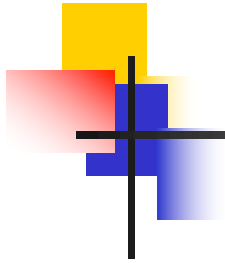


Componentes

- **ProductLink**

```
...  
const ProductLink = ({id, name}) => {  
  return (  
    <Link to={` /catalog/product-details/${id}`} >  
      {name}  
    </Link>  
  );  
}  
...
```

- Cuando el usuario hace clic sobre el nombre de un producto, se renderiza **ProductDetails**
 - [React Router] **Body** asocia la ruta `/catalog/product-details/:id` con **ProductDetails**



Componentes



ProductDetails

```
...
const ProductDetails = () => {
  const loggedIn = useSelector(users.selectors.isLoggedIn);
  const [product, setProduct] = useState(null);
  const {id} = useParams();
  ...
```

Estado local (la información detallada del producto sólo se necesita establecer y consultar desde ProductDetails)

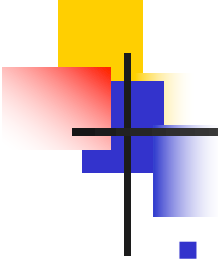
```
if (!product) {
  return null;
}
```

Si la respuesta del backend todavía no llegó, no se renderiza nada (`null`)

```
return (
```

```
...
<BackLink/>
... Imprimir detalles de product ...
... Incluir <AddToShoppingCart> si loggedIn ...
...
);
}
```

```
export default ProductDetails;
```



ProductDetails

- [1] Cuando **ProductDetails** se monta en el Virtual DOM, es decir, tras la primera renderización, el componente no mostrará nada (**product** es **null**)
- [2] Justo tras esa primera renderización, queremos hacer una petición al backend para obtener la información detallada del producto y almacenarla en el estado local (**setProduct**)
- [3] Como consecuencia de ese cambio de estado, el componente se volverá a renderizar y mostrará la información del producto (**product** contendrá la información detallada)

Hook useEffect

```
import {useState, useEffect} from 'react';
```

```
...
```

```
const ProductDetails = () => {
```

```
  const [product, setProduct] = useState(null);
```

```
  const {id} = useParams();
```

```
  const productId = Number(id);
```

```
  ...
```

```
  useEffect(() => {
```

```
    const findProductById = async productId => {
```

```
      if (!Number.isNaN(productId)) {
```

```
        const response = await
```

```
          backend.catalogService.findProductById(productId);
```

```
        if (response.ok) {
```

```
          setProduct(response.payload);
```

```
        }
```

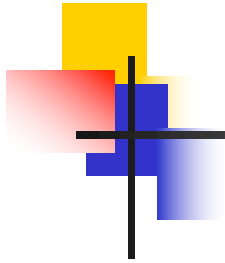
```
      }
```

```
    }
```

```
    findProductById(productId);
```

efecto

```
  }, [productId]);
```

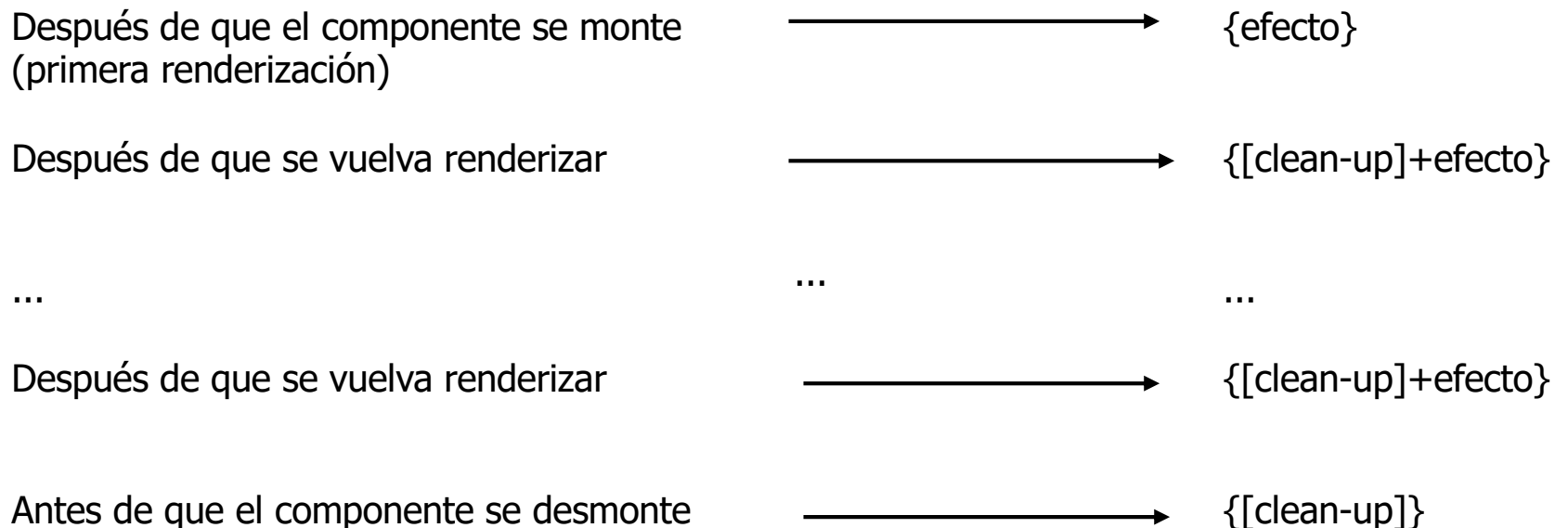



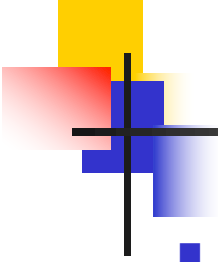
Hook useEffect

```
    if (!product) {  
        return null;  
    }  
  
    return (  
        ...  
        <BackLink/>  
        ... Imprimer detalles de product ...  
        ... Incluir <AddToShoppingCart> si loggedIn ...  
        ...  
    );  
}  
  
export default ProductDetails;
```

Hook useEffect

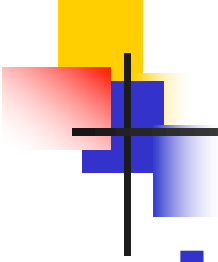
- El Hook `useEffect` recibe dos argumentos
 - [Obligatorio] **efecto**
 - [Opcional] **dependencias**
- Efecto
 - Es una función que se ejecuta justo después de cada renderización del componente
 - Opcionalmente puede devolver una función como resultado → a esta función le llamaremos función **clean-up** (limpieza)
- Modelo de ejecución





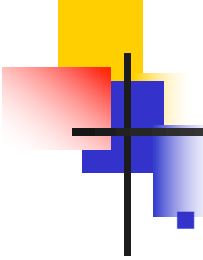
Hook useEffect

- El segundo argumento de `useEffect` (dependencias) es un array de valores y es opcional
- Este segundo argumento se puede usar para ejecutar `{[clean-up]+efecto}` condicionalmente tras cada nueva renderización
 - Si no se pasa este segundo argumento, siempre se ejecuta `{[clean-up]+efecto}` tras cada nueva renderización
 - Si se pasa un array vacío, no se ejecuta `{[clean-up]+efecto}` tras cada nueva renderización
 - Si se pasa un array no vacío, sólo se ejecutará `{[clean-up]+efecto}` tras cada nueva renderización si los valores recibidos son distintos a los de la última vez que se ejecutó el efecto



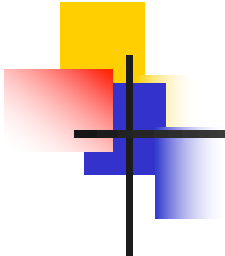
Hook useEffect – ProductDetails - dependencias

- Dependencias del efecto de **ProductDetails**
 - Variables usadas por el efecto, pero definidas fuera de él
 - `[productId]`
- Observación
 - Una vez que **ProductDetails** está montado en el Virtual DOM, `productId` nunca cambia



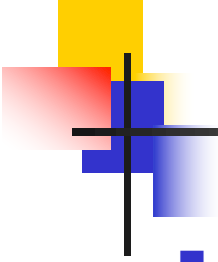
Hook useEffect – ProductDetails - ejecución

- [1] Cuando el componente se monta en el Virtual DOM, es decir, tras la primera renderización, el componente no mostrará nada (`product` es `null`)
- [2] Justo tras esa primera renderización, se ejecuta `{efecto}`
 - Se hace una petición al backend para obtener la información detallada del producto
`(backend.catalogService.findProductById(productId))`
 - Cuando llega la respuesta, la información detallada del producto se almacena en el estado local (`setProduct(response.payload)`)
- [3] Como consecuencia de ese cambio de estado, el componente se vuelve a renderizar y muestra los datos del producto (`product` contiene los datos del producto)
 - Como los valores de las dependencias no cambiaron, no se ejecuta `{[cleanup]+efecto}` tras esta renderización



Hook `useEffect` – ProductDetails - ejecución

- ¿Se podría haber pasado un array vacío como dependencias?
 - Sí
 - Pero no se hizo para evitar warnings producidos por el “linter”
 - El linter se puede ejecutar con `npm run lint`
 - El linter no sabe que `productId` nunca cambiará mientras el componente esté montado
 - La buena práctica es especificar todas las dependencias en `useEffect`
- Ejercicio: ¿se podría haber optado por no pasar el argumento con las dependencias?



React 18 y StrictMode

- Si se utiliza **StrictMode** (tema 5), como es el caso de los ejemplos de la asignatura, en **modo desarrollo** ocurre lo siguiente
 - Cada vez que se monta un componente (ejecución de {efecto}), se desmonta (ejecución de {[clean-up]}) y se vuelve a montar (ejecución de {efecto} otra vez)
 - Consecuencia: cuando se accede a la información detallada de un producto, la petición GET para obtener la información del producto se envía dos veces al backend
 - Sólo ocurre en modo desarrollo y no afecta a la semántica